PCPH 0001: CODING & PROGRAMMING APTITUDE

Introduction: This module has been designed keeping in mind the requirements of job seeking students and to prepare them to enhance their verbal ability so that they excel in a professional world, creating an impact through the right and appropriate usage of the skills of technical aptitude.

Objective1. Fundamental Data Structures Mastery:

- Understand and implement various data structures, including trees and graphs, using Java. Students will learn about binary trees, binary search trees, AVL trees, and graph traversals (BFS, DFS).

2. Algorithm Design and Analysis:

- Develop problem-solving skills by designing efficient algorithms for tree and graph-related problems. Students will analyze time and space complexities and learn about various algorithmic paradigms like greedy algorithms, backtracking, and divide and conquer.

3. Dynamic Programming Techniques:

- Gain proficiency in dynamic programming by solving a range of DP problems. Students will learn to identify overlapping subproblems and optimal substructure properties, enabling them to tackle complex problems in a structured manner.

4. LeetCode Problem Solving:

- Enhance practical coding skills by solving a curated set of LeetCode problems focused on trees, graphs, and dynamic programming. This hands-on approach will help students apply theoretical knowledge to realistic problems, preparing them for coding interviews.

5. Competitive Programming Strategies:

- Familiarize students with competitive programming strategies, including effective debugging techniques, time management during contests, and strategies for tackling unknown problems. This objective will cultivate a competitive mindset and improve overall coding proficiency.

Credits: 04 **Semester VI** **L–T–P-C: 4–0–0-0**

Module No.	Content	Teaching Hours (Approx.)
I	**Introduction to Trees** - Understanding tree structures - Binary trees: properties and basic operations - Binary Search Trees (BST): insertion, deletion, traversal **Tree Traversals and Recursion** - In-order, pre-order, post-order, level-order traversals - Implementing tree traversals in Java - Recursive vs. iterative techniques	12

	Trees in Coding Challenges - Solving LeetCode problems related to trees - Common tree algorithms: Lowest Common Ancestor (LCA), Diameter of a Binary Tree - Review and hands-on coding exercises	
II	**Introduction to Graphs** - Graph representation: adjacency matrix, adjacency list - Understanding directed vs. undirected graphs - Breadth-First Search (BFS) and Depth-First Search (DFS) **Graph Algorithms** - Shortest path algorithms: Dijkstra's, Bellman-Ford - Minimum spanning tree: Prim's and Kruskal's algorithms - Applications of graph algorithms in real-world scenarios **Dynamic Programming Concepts** - Introduction to dynamic programming: principles and approach - Common DP problems: Fibonacci sequence, Coin Change problem - Identifying subproblems and overlapping subproblems **Advanced DP Techniques and LeetCode Practice** - Advanced DP techniques: memoization vs. tabulation - Solving LeetCode problems involving DP, trees, and graphs - Review session and coding exercises	12

Reference Books/ Text Books / Cases:

- This syllabus provides a balanced approach to mastering trees, graphs, and dynamic programming, integrating theoretical knowledge with practical coding challenges. Each module builds upon the previous one, ensuring students develop a solid foundation and applicable skills in competitive programming.

Intended Outcomes:

This syllabus provides a balanced approach to mastering trees, graphs, and dynamic programming, integrating theoretical knowledge with practical coding challenges. Each module builds upon the previous one, ensuring students develop a solid foundation and applicable skills in competitive programming.